MongoDB Practical Solutions

Complete step-by-step guide for all MongoDB problem statements in the DBMS External Practical Question Bank

Includes: exact PDF question numbering, separate databases, fresh sample data, complete mongosh commands, expected results, and viva questions.

MongoDB Shell: mongosh

Study copy • Commands are intended to be run one question at a time

How to use this guide

1. Open MongoDB Shell (mongosh).
2. Go to the exact PDF question number you need.
3. Copy each command block and execute it separately, in order.
4. Run multiline commands as a complete block. Do not type the shell continuation prompt characters such as `|`.
5. Each question uses a separate database name so its data does not mix with the other questions.

Important: The commands use `db.collection.drop()` before inserting data. If the collection does not exist, MongoDB may return `false`; that is normal. Inserted ObjectId values are generated automatically and will differ on your computer.

Question numbers in the PDF

PDF number	Problem
2(B)	Product Inventory — Sorting and Limiting
5(B)	Student Database — CRUD Operations
12(B)	Delete and Remove Documents
13(B)	Aggregation Using $match, $project, and $sort
14(B)	Hospital Patient Records
15(B)	Array Fields and Array Queries
16(B)	MongoDB Indexing — Online Shopping Database
17(B)	Library Collection — Update Operations
18(B)	Integrated MongoDB Application — College Admission System
19(B)	Department-Wise Employee Aggregation
20(B)	Sales Analysis Using Aggregation

Question 2(B) — Product Inventory — Sorting and Limiting

PDF location: Page 1

Problem statement (from question bank): Create a products collection containing product ID, name, category, price, and stock. Display products costing above ■1,000, sort by price in descending order, display the top three expensive products, and use projection to display only product name and price.

Step 1: Select a new database

```
use ProductInventoryDB
```

Step 2: Reset the collection

```
db.products.drop()
```

If this returns false, the collection did not exist yet; continue.

Step 3: Insert six products

```
db.products.insertMany([
  { productId: 101, name: "Laptop", category: "Electronics", price: 55000, stock: 10 },
  { productId: 102, name: "Mobile", category: "Electronics", price: 25000, stock: 20 },
  { productId: 103, name: "Headphones", category: "Accessories", price: 1500, stock: 30 },
  { productId: 104, name: "Keyboard", category: "Accessories", price: 1200, stock: 15 },
  { productId: 105, name: "Mouse", category: "Accessories", price: 800, stock: 40 },
  { productId: 106, name: "Monitor", category: "Electronics", price: 12000, stock: 8 }
])
```

Step 4: Display all products

```
db.products.find()
```

Step 5: Find products costing above ■1,000

```
db.products.find({ price: { $gt: 1000 } })
```

Expected: Laptop, Mobile, Headphones, Keyboard, and Monitor (5 products).

Step 6: Sort by price in descending order

```
db.products.find().sort({ price: -1 })
```

Expected order:

Product	Price (■)
Laptop	55,000
Mobile	25,000
Monitor	12,000
Headphones	1,500
Keyboard	1,200
Mouse	800

Step 7: Display the top three expensive products

```
db.products.find().sort({ price: -1 }).limit(3)
```

Expected: Laptop, Mobile, Monitor.

Step 8: Projection — display only name and price

```
db.products.find(
  {},
  { _id: 0, name: 1, price: 1 }
)
```

The _id field is hidden; only name and price are shown.

Viva questions

Q. What does find() do?
Answer: Retrieves documents from a collection.

Q. What does $gt mean?
Answer: Greater than.

Q. What does sort({ price: -1 }) do?
Answer: Sorts by price in descending order.

Q. What does limit(3) do?
Answer: Returns at most three documents.

Q. What is projection?
Answer: It selects which fields are included or excluded in the output.

Question 5(B) — Student Database — CRUD Operations

PDF location: Page 2

Problem statement (from question bank): Create a database CollegeDB and collection students. Insert at least five student documents containing roll number, name, department, and marks. Display all documents, find students scoring above 75, update a student's marks, and delete a specified student.

Step 1: Select database

```
use CollegeDB
```

Step 2: Reset collection

```
db.students.drop()
```

Step 3: Insert five students

```
db.students.insertMany([
  { rollNo: 1, name: "Amit", department: "IT", marks: 85 },
  { rollNo: 2, name: "Priya", department: "CS", marks: 92 },
  { rollNo: 3, name: "Rahul", department: "IT", marks: 68 },
  { rollNo: 4, name: "Sneha", department: "CS", marks: 78 },
  { rollNo: 5, name: "Vijay", department: "ENTC", marks: 55 }
])
```

Step 4: Display all students

```
db.students.find()
```

Step 5: Find students scoring above 75

```
db.students.find({ marks: { $gt: 75 } })
```

Roll No.	Name	Marks
1	Amit	85
2	Priya	92
4	Sneha	78

Step 6: Update Rahul's marks from 68 to 80

```
db.students.updateOne(
  { rollNo: 3 },
  { $set: { marks: 80 } }
)
```

Step 7: Verify Rahul's updated record

```
db.students.find({ rollNo: 3 })
```

Step 8: Delete Vijay

```
db.students.deleteOne({ rollNo: 5 })
```

Step 9: Display remaining students

```
db.students.find()
```

Expected: 4 student documents remain.

Viva questions

Q. What does CRUD stand for?
Answer: Create, Read, Update, Delete.

Q. What does $set do?
Answer: Sets a field to a specified value or creates it if absent.

Q. updateOne() vs updateMany()?
Answer: updateOne updates at most one matching document; updateMany updates all matching documents.

Q. What does deleteOne() do?
Answer: Deletes the first document matching the filter.

Question 12(B) — Delete and Remove Documents

PDF location: Page 5

Problem statement (from question bank): Create a products collection. Demonstrate deleteOne() to remove a specified product and deleteMany() to remove products matching a given condition. Display remaining documents and explain the difference between deleting documents and dropping a collection.

Step 1: Select database

```
use DeleteProductsDB
```

Step 2: Reset collection

```
db.products.drop()
```

Step 3: Insert sample products

```
db.products.insertMany([
  { productId: 101, name: "Laptop", category: "Electronics", price: 55000, stock: 10 },
  { productId: 102, name: "Mobile", category: "Electronics", price: 25000, stock: 20 },
  { productId: 103, name: "Headphones", category: "Accessories", price: 1500, stock: 30 },
  { productId: 104, name: "Keyboard", category: "Accessories", price: 1200, stock: 15 },
  { productId: 105, name: "Mouse", category: "Accessories", price: 800, stock: 40 },
  { productId: 106, name: "Monitor", category: "Electronics", price: 12000, stock: 8 }
])
```

Step 4: Display all products

```
db.products.find()
```

Step 5: Delete one specified product (Mouse)

```
db.products.deleteOne({ productId: 105 })
```

Expected: deletedCount: 1.

Step 6: Delete all remaining Accessories products

```
db.products.deleteMany({ category: "Accessories" })
```

Expected: deletedCount: 2, because Mouse was already deleted; Headphones and Keyboard are removed now.

Step 7: Display remaining documents

```
db.products.find()
```

Product ID	Name	Category	Price (■)
101	Laptop	Electronics	55,000
102	Mobile	Electronics	25,000
106	Monitor	Electronics	12,000

Step 8: Drop the entire collection (run after checking remaining documents)

```
db.products.drop()
```

Expected: true if the collection existed.

Command	Effect
deleteOne(filter)	Deletes the first matching document.
deleteMany(filter)	Deletes all documents matching the filter.
db.products.drop()	Removes the collection and its indexes.

Viva questions

Q. What happens if no document matches a delete filter?
Answer: deletedCount is 0.

Q. Difference between deleteMany({}) and drop()?
Answer: deleteMany({}) removes all documents but keeps the collection; drop() removes the collection itself.

Question 13(B) — Aggregation Using $match, $project, and $sort

PDF location: Page 5

Problem statement (from question bank): Create an employees collection. Use an aggregation pipeline to filter employees earning more than ■30,000, project only employee name and salary, and sort the output by salary in descending order.

Step 1: Select database

```
use EmployeeAggregationDB
```

Step 2: Reset collection

```
db.employees.drop()
```

Step 3: Insert six employees

```
db.employees.insertMany([
  { empId: 101, empName: "Amit", department: "IT", salary: 45000 },
  { empId: 102, empName: "Priya", department: "HR", salary: 35000 },
  { empId: 103, empName: "Rahul", department: "IT", salary: 28000 },
  { empId: 104, empName: "Sneha", department: "Finance", salary: 60000 },
  { empId: 105, empName: "Vijay", department: "HR", salary: 32000 },
  { empId: 106, empName: "Neha", department: "IT", salary: 50000 }
])
```

Step 4: Display all employees

```
db.employees.find()
```

Step 5: Run the aggregation pipeline

```
db.employees.aggregate([
  {
    $match: {
      salary: { $gt: 30000 }
    }
  },
  {
    $project: {
      _id: 0,
      empName: 1,
      salary: 1
    }
  },
  {
    $sort: {
      salary: -1
    }
  }
])
```

Expected result (5 employees; Rahul is excluded because his salary is ■28,000):

Employee	Salary (■)
Sneha	60,000
Neha	50,000
Amit	45,000
Priya	35,000
Vijay	32,000

Viva questions

Q. What is an aggregation pipeline?
Answer: A sequence of stages that process documents to produce a result.

Q. What does $match do?
Answer: Filters documents.

Q. What does $project do?
Answer: Selects or reshapes fields in output documents.

Q. What does $sort: { salary: -1 } do?
Answer: Sorts salaries from highest to lowest.

Question 14(B) — Hospital Patient Records

PDF location: Page 6

Problem statement (from question bank): Create a patients collection containing patient ID, name, age, disease, and doctor. Insert at least five documents. Retrieve patients above 60 years, filter by a specified disease, and count patients for each disease using aggregation.

Step 1: Select database

```
use HospitalDB
```

Step 2: Reset collection

```
db.patients.drop()
```

Step 3: Insert five patients

```
db.patients.insertMany([
  { patientId: 101, name: "Amit", age: 65, disease: "Diabetes", doctor: "Dr. Sharma" },
  { patientId: 102, name: "Priya", age: 45, disease: "Asthma", doctor: "Dr. Patil" },
  { patientId: 103, name: "Rahul", age: 72, disease: "Diabetes", doctor: "Dr. Sharma" },
  { patientId: 104, name: "Sneha", age: 55, disease: "Fever", doctor: "Dr. Joshi" },
  { patientId: 105, name: "Vijay", age: 68, disease: "Asthma", doctor: "Dr. Patil" }
])
```

Step 4: Display all patient records

```
db.patients.find()
```

Step 5: Find patients above 60 years

```
db.patients.find({ age: { $gt: 60 } })
```

Expected: Amit (65), Rahul (72), and Vijay (68).

Step 6: Find patients with Diabetes

```
db.patients.find({ disease: "Diabetes" })
```

Expected: Amit and Rahul.

Step 7: Count patients per disease

```
db.patients.aggregate([
  {
    $group: {
      _id: "$disease",
      totalPatients: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } }
])
```

Disease	Total patients
Asthma	2
Diabetes	2
Fever	1

Viva questions

Q. What is a collection?
Answer: A group of MongoDB documents, broadly similar to a table in a relational database.

Q. What does $group do?
Answer: Groups documents and calculates aggregate results.

Q. What does $sum: 1 count?
Answer: Adds one for each document in a group.

Q. What does _id: "$disease" mean?
Answer: Groups documents by their disease value.

Question 15(B) — Array Fields and Array Queries

PDF location: Page 6

Problem statement (from question bank): Create a students collection with a skills array. Insert documents containing different skills. Retrieve students having a specified skill using $in or array matching, and update a student's array using $push or $addToSet.

Step 1: Select database

```
use StudentSkillsDB
```

Step 2: Reset collection

```
db.students.drop()
```

Step 3: Insert five students with skills arrays

```
db.students.insertMany([
  { rollNo: 1, name: "Amit", skills: ["Java", "Python", "MongoDB"] },
  { rollNo: 2, name: "Priya", skills: ["Python", "C++", "HTML"] },
  { rollNo: 3, name: "Rahul", skills: ["Java", "React", "Node.js"] },
  { rollNo: 4, name: "Sneha", skills: ["MongoDB", "Python", "SQL"] },
  { rollNo: 5, name: "Vijay", skills: ["C++", "HTML", "CSS"] }
])
```

Step 4: Display all students

```
db.students.find()
```

Step 5: Find students with Python

```
db.students.find({ skills: "Python" })
```

Expected: Amit, Priya, Sneha.

Step 6: Find students with Java or MongoDB

```
db.students.find({
  skills: { $in: ["Java", "MongoDB"] }
})
```

Expected: Amit, Rahul, Sneha.

Step 7: Add Docker to Priya's skills using $push

```
db.students.updateOne(
  { rollNo: 2 },
  { $push: { skills: "Docker" } }
)
```

Step 8: Verify Priya's record

```
db.students.find({ rollNo: 2 })
```

Step 9: Add Python to Rahul's skills using $addToSet

```
db.students.updateOne(
  { rollNo: 3 },
  { $addToSet: { skills: "Python" } }
)
```

Step 10: Verify Rahul's record

```
db.students.find({ rollNo: 3 })
```

Viva questions

Q. What is an array field?
Answer: A field that stores multiple values in one document.

Q. What does $push do?
Answer: Adds an element to an array and allows duplicates.

Q. What does $addToSet do?
Answer: Adds an element only if it is not already present.

Q. What does $in do?
Answer: Matches a field against one or more values in a specified list.

Question 16(B) — MongoDB Indexing — Online Shopping Database

PDF location: Page 6

Problem statement (from question bank): Create a products collection containing product ID, name, category, price, and stock. Create suitable single-field, compound, and unique indexes. Execute queries using indexed fields and demonstrate index inspection using getIndexes() and query analysis using explain().

Step 1: Select database

```
use OnlineShoppingDB
```

Step 2: Reset collection

```
db.products.drop()
```

Step 3: Insert five products

```
db.products.insertMany([
  { productId: 101, name: "Laptop", category: "Electronics", price: 55000, stock: 10 },
  { productId: 102, name: "Mobile", category: "Electronics", price: 25000, stock: 20 },
  { productId: 103, name: "Headphones", category: "Accessories", price: 1500, stock: 30 },
  { productId: 104, name: "Keyboard", category: "Accessories", price: 1200, stock: 15 },
  { productId: 105, name: "Monitor", category: "Electronics", price: 12000, stock: 8 }
])
```

Step 4: Create a single-field index

```
db.products.createIndex({ category: 1 })
```

Expected index name: category_1.

Step 5: Create a compound index

```
db.products.createIndex({ category: 1, price: -1 })
```

Expected index name: category_1_price_-1.

Step 6: Create a unique index

```
db.products.createIndex({ productId: 1 }, { unique: true })
```

Expected index name: productId_1.

Step 7: Query using indexed fields

```
db.products.find({ category: "Electronics" })
db.products.find({ category: "Electronics" }).sort({ price: -1 })
```

Step 8: Inspect indexes

```
db.products.getIndexes()
```

Expected indexes include _id_, category_1, category_1_price_-1, and productId_1.

Step 9: Analyse query execution

```
db.products.find({ category: "Electronics" }).explain("executionStats")
```

Inspect executionStats, nReturned, totalDocsExamined, and executionStages. MongoDB may use IXSCAN or COLLSCAN depending on its query plan.

Viva questions

Q. What is an index?
Answer: A data structure that can help MongoDB find documents more efficiently.

Q. What is a unique index?
Answer: It prevents duplicate values for the indexed key, subject to MongoDB's index rules.

Q. What is a compound index?
Answer: An index containing more than one field.

Q. What does getIndexes() do?
Answer: Lists indexes on a collection.

Q. What does explain() do?
Answer: Shows the query execution plan and execution statistics.

Question 17(B) — Library Collection — Update Operations

PDF location: Page 7

Problem statement (from question bank): Create a books collection containing ISBN, title, author, category, and price. Insert at least five documents. Use updateOne() to modify one book, updateMany() to update a category of books, and $inc to increase a specified price or stock value.

Step 1: Select database

```
use LibraryUpdateDB
```

Step 2: Reset collection

```
db.books.drop()
```

Step 3: Insert five books

```
db.books.insertMany([
  { ISBN: "B101", title: "DBMS", author: "Korth", category: "Education", price: 500 },
  { ISBN: "B102", title: "Java", author: "James", category: "Programming", price: 600 },
  { ISBN: "B103", title: "Python", author: "Guido", category: "Programming", price: 450 },
  { ISBN: "B104", title: "C++", author: "Bjarne", category: "Programming", price: 550 },
  { ISBN: "B105", title: "Networking", author: "Tanenbaum", category: "Education", price: 700 }
])
```

Step 4: Display all books

```
db.books.find()
```

Step 5: Update DBMS price to ■550 using updateOne()

```
db.books.updateOne(
  { ISBN: "B101" },
  { $set: { price: 550 } }
)
```

Step 6: Change Programming category using updateMany()

```
db.books.updateMany(
  { category: "Programming" },
  { $set: { category: "Computer Science" } }
)
```

Expected: matchedCount: 3 and modifiedCount: 3.

Step 7: Increase Python price by ■50 using $inc

```
db.books.updateOne(
  { ISBN: "B103" },
  { $inc: { price: 50 } }
)
```

Step 8: Display final books

```
db.books.find()
```

ISBN	Title	Category	Price (■)
B101	DBMS	Education	550
B102	Java	Computer Science	600
B103	Python	Computer Science	500
B104	C++	Computer Science	550
B105	Networking	Education	700

Viva questions

Q. What does updateOne() do?
Answer: Updates one matching document.

Q. What does updateMany() do?
Answer: Updates all matching documents.

Q. What does $inc do?
Answer: Increases or decreases a numeric field by the specified amount.

Q. What does $set do?
Answer: Sets a field to a specified value.

Question 18(B) — Integrated MongoDB Application — College Admission System

PDF location: Page 7

Problem statement (from question bank): Create an admissions collection containing application number, student name, course, category, entrance score, and admission status. Perform CRUD operations, retrieve records using logical and comparison operators, create indexes for efficient searching, and use aggregation pipelines to calculate course-wise application counts and average entrance scores. Demonstrate suitable update and deletion operations.

Step 1: Select database

```
use DBMS_Practical
```

Step 2: Reset collection

```
db.admissions.drop()
```

Step 3: Insert five admission records

```
db.admissions.insertMany([
  { applicationNo: 1001, studentName: "Amit", course: "BTech IT", category: "OPEN", entranceScore: 85, admissionStatus: "A
  { applicationNo: 1002, studentName: "Priya", course: "BTech CS", category: "OBC", entranceScore: 92, admissionStatus: "A
  { applicationNo: 1003, studentName: "Rahul", course: "BTech IT", category: "SC", entranceScore: 65, admissionStatus: "Pe
  { applicationNo: 1004, studentName: "Sneha", course: "BTech CS", category: "OPEN", entranceScore: 78, admissionStatus: '
  { applicationNo: 1005, studentName: "Vijay", course: "BTech IT", category: "OBC", entranceScore: 55, admissionStatus: "P
])
```

Step 4: Display all records

```
db.admissions.find()
```

Step 5: Comparison operator — score above 80

```
db.admissions.find({ entranceScore: { $gt: 80 } })
```

Expected: Amit and Priya.

Step 6: Logical $or — OPEN category or score above 90

```
db.admissions.find({
  $or: [
    { category: "OPEN" },
    { entranceScore: { $gt: 90 } }
  ]
})
```

Expected: Amit, Priya, Sneha.

Step 7: Logical $and — BTech IT and score above 60

```
db.admissions.find({
  $and: [
    { course: "BTech IT" },
    { entranceScore: { $gt: 60 } }
  ]
})
```

Expected: Amit and Rahul.

Step 8: Create a unique application number index

```
db.admissions.createIndex(
  { applicationNo: 1 },
  { unique: true }
)
```

Step 9: Create course index and inspect indexes

```
db.admissions.createIndex({ course: 1 })
db.admissions.getIndexes()
```

Step 10: Calculate course-wise application count and average score

```
db.admissions.aggregate([
  {
    $group: {
      _id: "$course",
```

```
      totalApplications: { $sum: 1 },
      averageScore: { $avg: "$entranceScore" }
    }
  },
  { $sort: { _id: 1 } }
])
```

Course	Applications	Average score
BTech CS	2	85
BTech IT	3	68.33333333333333

Step 11: Update Rahul's status to Admitted

```
db.admissions.updateOne(
  { applicationNo: 1003 },
  { $set: { admissionStatus: "Admitted" } }
)
```

Step 12: Verify Rahul's record

```
db.admissions.find({ applicationNo: 1003 })
```

Step 13: Delete Vijay's application

```
db.admissions.deleteOne({ applicationNo: 1005 })
```

Step 14: Display final records

```
db.admissions.find()
```

Expected: 4 documents remain.

Viva questions

Q. What is CRUD?
Answer: Create, Read, Update, Delete.

Q. What does $gt mean?
Answer: Greater than.

Q. What do $or and $and do?
Answer: $or matches if at least one condition is true; $and requires all conditions to be true.

Q. Why create a unique index?
Answer: To prevent duplicate application numbers.

Q. What does $avg calculate?
Answer: The average of numeric values in each aggregation group.

Question 19(B) — Department-Wise Employee Aggregation

PDF location: Page 8

Problem statement (from question bank): Create an employees collection. Use the aggregation framework with $group to calculate the number of employees and average salary in each department. Sort the output by average salary.

Step 1: Select database

```
use DepartmentAggregationDB
```

Step 2: Reset collection

```
db.employees.drop()
```

Step 3: Insert six employees

```
db.employees.insertMany([
  { empId: 101, empName: "Amit", department: "IT", salary: 50000 },
  { empId: 102, empName: "Priya", department: "HR", salary: 35000 },
  { empId: 103, empName: "Rahul", department: "IT", salary: 60000 },
  { empId: 104, empName: "Sneha", department: "HR", salary: 45000 },
  { empId: 105, empName: "Vijay", department: "Finance", salary: 40000 },
  { empId: 106, empName: "Neha", department: "Finance", salary: 50000 }
])
```

Step 4: Display employees

```
db.employees.find()
```

Step 5: Group by department and sort by average salary

```
db.employees.aggregate([
  {
    $group: {
      _id: "$department",
      totalEmployees: { $sum: 1 },
      averageSalary: { $avg: "$salary" }
    }
  },
  { $sort: { averageSalary: 1 } }
])
```

Department	Employees	Average salary (₹)
HR	2	40,000
Finance	2	45,000
IT	2	55,000

To sort from highest average salary to lowest, use `{ $sort: { averageSalary: -1 } }`.

Viva questions

Q. What does $sum: 1 do in $group?
Answer: Counts each document as one.

Q. What does $avg do?
Answer: Calculates the average numeric value.

Q. What does $sort: { averageSalary: 1 } do?
Answer: Sorts average salaries in ascending order.

Question 20(B) — Sales Analysis Using Aggregation

PDF location: Page 8

Problem statement (from question bank): Create a sales collection with product, category, quantity, unit price, and date. Use $group and $sum to calculate total quantity and revenue per category, and use $sort to rank categories by revenue.

Step 1: Select database

```
use SalesAggregationDB
```

Step 2: Reset collection

```
db.sales.drop()
```

Step 3: Insert six sales records

```
db.sales.insertMany([
  { product: "Laptop", category: "Electronics", quantity: 2, unitPrice: 50000, date: new Date("2026-01-10") },
  { product: "Mobile", category: "Electronics", quantity: 3, unitPrice: 20000, date: new Date("2026-01-12") },
  { product: "Keyboard", category: "Accessories", quantity: 5, unitPrice: 1000, date: new Date("2026-01-15") },
  { product: "Mouse", category: "Accessories", quantity: 10, unitPrice: 500, date: new Date("2026-01-18") },
  { product: "Monitor", category: "Electronics", quantity: 2, unitPrice: 10000, date: new Date("2026-01-20") },
  { product: "Headphones", category: "Accessories", quantity: 4, unitPrice: 2000, date: new Date("2026-01-22") }
])
```

Step 4: Display sales records

```
db.sales.find()
```

Step 5: Calculate quantity and revenue by category; rank by revenue

```
db.sales.aggregate([
  {
    $group: {
      _id: "$category",
      totalQuantity: { $sum: "$quantity" },
      totalRevenue: {
        $sum: { $multiply: ["$quantity", "$unitPrice"] }
      }
    }
  },
  { $sort: { totalRevenue: -1 } }
])
```

Category	Total quantity	Total revenue (■)
Electronics	7	180,000
Accessories	19	18,000

Formula: Revenue for each sale = quantity × unit price. The aggregation adds each sale's revenue within its category.

Viva questions

Q. What does $multiply do?
Answer: Multiplies numeric expressions.

Q. What does $sum do?
Answer: Adds values within each group.

Q. How is total revenue calculated?
Answer: Sum of quantity multiplied by unit price for all sales in the category.

Q. What does $sort: { totalRevenue: -1 } do?
Answer: Sorts categories from highest revenue to lowest.

Last-minute practical checklist

• Always select the intended database with `use DatabaseName`.
• Run each full command block separately and wait for the result before moving on.
• Use `find()` to verify inserted or updated records.
• MongoDB automatically adds an `_id` field unless otherwise specified.
• The exact ObjectId values and some shell formatting will differ on your computer.
• If a collection is reset with `drop()`, its documents and indexes are removed. Create the indexes again when that question requires them.
• For question 12(B), run the final `db.products.drop()` only after displaying the remaining documents.

Core MongoDB operators used

Operator / method	Purpose
find()	Read matching documents
insertMany()	Insert multiple documents
updateOne() / updateMany()	Update one / all matching documents
deleteOne() / deleteMany()	Delete one / all matching documents
$gt	Greater than
$or / $and	Logical conditions
$set	Set a field value
$inc	Increment/decrement numeric value
$push / $addToSet	Add array element / add only if absent
$match / $project / $sort	Filter / select fields / sort
$group / $sum / $avg	Group documents and calculate totals / averages
createIndex() / getIndexes() / explain()	Create, inspect, and analyse indexes